

Arquitectura del Computador

Práctica de Laboratorio 3

Angel Navas y Jose Natera
Departamento de Computación
Universidad de Carabobo
Barbula, Carabobo

Abstract—Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial atención a operaciones de entrada/salida (E/S) mediante el manejo de dispositivos de hardware mapeados en memoria.

Index Terms—Computer Architecture, Assembly Language, I/O Operations, Hardware Simulation

I. INFORME

A. *Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?*

En el modelo de arquitectura MIPS32 implementado en simuladores como MARS, la organización de la memoria bajo el esquema de memory-mapped I/O (MMIO) elimina la necesidad de instrucciones especiales para la entrada y salida de datos, integrando los periféricos directamente en el espacio de direccionamiento lógico del procesador. En este diseño, los dispositivos de hardware no poseen un bus independiente, sino que se les asignan direcciones específicas dentro de una región reservada que generalmente comienza en el segmento de External I/O, situado típicamente a partir de la dirección 0xffff0000. Al segmentar la memoria de esta manera, el sistema operativo y el programador pueden interactuar con registros de control y de datos simplemente accediendo a estas locaciones de memoria fija, lo que permite que el hardware responda de forma síncrona o mediante interrupciones según el estado de los bits en dichas direcciones. Esta integración tiene implicaciones directas en el uso de las instrucciones de carga y almacenamiento, ya que las operaciones tradicionales lw y sw se transforman en las herramientas fundamentales para la comunicación con el exterior. Cuando el procesador ejecuta un sw hacia una dirección mapeada a un dispositivo, en lugar de modificar una celda de RAM convencional, activa un bus que envía la información al periférico correspondiente, como mostrar un carácter en la consola o mover un cursor. Por el contrario, un lw desde estas direcciones permite monitorear el estado del hardware, verificando si un dato está listo para ser leído; esto significa que el programador debe ser extremadamente cuidadoso con la sincronización, asegurándose de que el dispositivo esté preparado antes de realizar la transferencia para evitar la pérdida de información o estados de espera indefinidos.

B. *¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?*

La diferencia fundamental radica en cómo la CPU percibe los periféricos: mientras que el MMIO trata a los dispositivos como si fueran simples direcciones de memoria, la entrada/salida por puertos (Isolated I/O) utiliza un espacio de direccionamiento separado y exclusivo, accesible solo mediante instrucciones específicas de bajo nivel como IN o OUT. En MMIO, el procesador no distingue entre escribir en una variable en RAM o enviar datos a una pantalla, lo que simplifica enormemente el diseño del hardware y permite reutilizar todas las instrucciones de manipulación de datos existentes. Por el contrario, el enfoque por puertos protege la memoria principal al aislar el tráfico de periféricos, pero obliga a los desarrolladores a aprender un conjunto de instrucciones adicional y limita la flexibilidad de los modos de direccionamiento disponibles para interactuar con el hardware. MIPS32 adopta predominantemente MMIO debido a su filosofía de arquitectura de conjunto de instrucciones reducido (RISC), que busca minimizar la complejidad de la unidad de control eliminando instrucciones especializadas innecesarias. Al mapear los dispositivos en el rango superior de la memoria, como se observa en la región de External I/O, el sistema gana una ventaja significativa en términos de ortogonalidad: cualquier operación que funcione sobre la memoria funciona también sobre los dispositivos. Sin embargo, esto conlleva la desventaja de robar espacio del mapa de direcciones lógicas y la necesidad de gestionar cuidadosamente el almacenamiento en caché para evitar que la CPU lea datos obsoletos de un registro de hardware que cambia constantemente. En el contexto de MARS, esto facilita que los estudiantes entiendan el flujo de datos sin salir del paradigma de registros y memoria convencional.

C. *En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?*

El principal conflicto que surge cuando dos dispositivos comparten o solapan sus direcciones de memoria es la colisión de bus y la ambigüedad en la integridad de los datos, un fenómeno donde el procesador pierde la capacidad de comunicarse de forma unívoca con el hardware. Si dos periféricos responden simultáneamente a una instrucción sw,

ambos intentarían procesar la señal, lo que podría derivar en comportamientos erráticos del sistema o, en casos de hardware real, daños físicos por cortocircuitos si ambos dispositivos intentan controlar las líneas de datos con voltajes opuestos. En el caso de una operación `lw`, el procesador recibiría señales de múltiples fuentes a la vez, resultando en un valor basura que no representa el estado real de ninguno de los componentes, invalidando cualquier lógica de control basada en banderas de estado o registros de datos. Para evitar este caos, los sistemas utilizan una unidad lógica denominada decodificador de direcciones, la cual actúa como un árbitro de tráfico que garantiza la exclusividad en el acceso al bus. Este componente electrónico analiza los bits de dirección emitidos por la CPU y genera una señal de selección de chip única para cada dispositivo, asegurando que solo un periférico esté activo y conectado al bus de datos en un instante dado. En entornos de simulación como MIPS32 en MARS, este conflicto se previene mediante una segmentación estricta del mapa de memoria, donde el diseño de la arquitectura reserva rangos fijos e inamovibles para cada controlador, eliminando la posibilidad de que un programador mapee accidentalmente dos piezas de hardware en la misma coordenada lógica.

D. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

El uso de MMIO simplifica el diseño del conjunto de instrucciones al adherirse al principio de ortogonalidad, donde las instrucciones de transferencia de datos existentes, como `lw`, `sw`, `lb` o `sb`, son suficientes para controlar cualquier periférico. Al tratar los registros de los dispositivos como celdas de memoria estándar, el hardware del procesador no necesita una unidad de control compleja que distinga entre diferentes espacios de direccionamiento, lo que reduce la cantidad de transistores y rutas de datos necesarias en el silicio. Esta uniformidad permite que todas las potentes herramientas de manipulación de datos, como las operaciones lógicas, los desplazamientos de bits y los diversos modos de direccionamiento, se apliquen directamente sobre los registros de E/S sin necesidad de mover los datos previamente a un registro de propósito general para procesarlos. Si un sistema optara por el esquema de E/S por puertos, el procesador se vería obligado a incluir instrucciones especializadas, comúnmente denominadas `IN` y `OUT`, que poseen su propio formato y lógica de ejecución dentro del ciclo de instrucción. Estas instrucciones adicionales requerirían una señal de control dedicada en el bus del sistema para indicar que la dirección emitida pertenece al espacio de E/S y no a la memoria RAM, lo que complica el diseño de la ALU y la unidad de decodificación. Además, se perdería flexibilidad, ya que estas instrucciones suelen estar limitadas a transferencias simples entre un registro específico y el puerto, careciendo de la capacidad de realizar operaciones complejas directamente sobre el periférico, lo que resultaría en un código de ensamblador más extenso y menos eficiente para tareas de monitoreo constante.

E. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

Cuando el procesador emite una dirección de memoria a través del bus de direcciones, el hardware no distingue intrínsecamente entre una celda de RAM y un registro de un periférico; en su lugar, el sistema depende de un componente crítico llamado decodificador de direcciones. Este circuito monitorea constantemente los bits de mayor peso en el bus y, al detectar un patrón que coincide con el rango reservado para Memory-Mapped I/O (como el prefijo `0xffff` en MIPS32), desactiva la señal de habilitación de la memoria principal y activa la línea de selección del controlador del dispositivo correspondiente. A nivel del bus de datos, esto establece un puente directo: si la operación es un `sw`, los datos viajan desde los registros del procesador hacia los buffers del periférico; si es un `lw`, el dispositivo coloca su estado interno en el bus para que el procesador lo capture, todo esto ocurriendo de manera transparente para la ALU. La diferenciación entre el acceso a la RAM o a un periférico se consolida mediante el uso de señales de control sincronizadas con el ciclo de reloj, donde el mapa de memoria actúa como el plano maestro del sistema. En arquitecturas como MIPS32, este diseño asegura que no existan colisiones en el bus de sistema, ya que la lógica combinacional del decodificador garantiza que solo un esclavo (ya sea un módulo de memoria o un controlador de E/S) responda a la maestra (la CPU) en un instante dado. Esta arquitectura de bus compartido simplifica la topología del hardware, permitiendo que el procesador utilice el mismo protocolo de lectura y escritura para encender un LED, leer una tecla o almacenar una variable, delegando la responsabilidad de la ruta física a la lógica de direccionamiento externa al núcleo.

F. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

La capacidad de un programa para acceder a dispositivos mediante Memory-Mapped I/O depende enteramente del modo de ejecución del procesador y de la configuración del Mapa de Memoria. Generalmente, los registros de control de hardware se ubican en el segmento de Kernel Address Space (direcciones superiores a `0x80000000`), lo que significa que un programa de usuario estándar que se ejecuta en modo usuario no puede acceder a estas direcciones directamente. Si un proceso sin privilegios intenta ejecutar un `lw` o `sw` sobre una dirección protegida como `0xffff0000`, la Unidad de Gestión de Memoria (MMU) detecta una violación de acceso y genera una excepción de dirección no válida, deteniendo la ejecución del programa y transfiriendo el control al sistema operativo para proteger la integridad del hardware. Para evitar estos accesos no autorizados, el mecanismo de protección principal es la paginación y segmentación gestionada por la MMU. Este componente actúa como una aduana que traduce direcciones virtuales a físicas, verificando los bits de permiso de lectura, escritura o ejecución asociados a cada página de memoria según el nivel de privilegio actual del procesador. Además, el

sistema operativo implementa syscalls como el único puente seguro; en lugar de permitir que el usuario manipule el hardware, el programa solicita al kernel que realice la operación de E/S en su nombre. Esto garantiza que el acceso a periféricos críticos sea ordenado, seguro y que ningún proceso malicioso o mal programado pueda desestabilizar el sistema operativo interfiriendo directamente con los registros de control del bus.

G. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

Para evitar el desperdicio de ciclos de reloj que supone el polling o espera activa, la técnica más eficiente es el uso de E/S dirigida por interrupciones. En este esquema, el procesador no consulta constantemente el bit de estado de un dispositivo en el segmento de Memory-Mapped I/O, sino que continúa ejecutando otras tareas de cómputo útil hasta que el periférico envía una señal eléctrica a través de una línea de interrupción dedicada. Al recibir esta señal, el hardware del procesador suspende la ejecución actual, guarda el estado de los registros y salta a una dirección específica del kernel conocida como Manejador de Interrupciones, donde se procesa el dato recibido antes de retomar el programa principal. Otra técnica avanzada para el manejo de grandes volúmenes de datos, como en discos o tarjetas de red, es el Acceso Directo a Memoria (DMA). En este escenario, un controlador de DMA asume temporalmente el control del bus de sistema para transferir bloques completos de información entre el periférico y la memoria RAM sin intervención constante de la CPU. Mientras el DMA gestiona la transferencia, el procesador queda totalmente libre para ejecutar hilos de usuario, recibiendo una única interrupción al finalizar toda la operación de bloque. En simuladores como MARS, aunque el DMA es menos común, se enfatiza el uso de excepciones de hardware para que el estudiante aprenda a programar sistemas reactivos que solo consumen recursos de procesamiento cuando existe una interacción real con el entorno.

H. Análisis y Discusión de los Resultados

El análisis de los ejercicios resueltos revela la naturaleza jerárquica y secuencial de la interacción con el hardware mediante Memory-Mapped I/O. En cada caso, el procesador MIPS32 debe seguir un protocolo de tres pasos: activación mediante un comando en el registro de control, sincronización mediante una espera activa sobre el registro de estado, y finalmente la extracción de datos de los registros correspondientes. La diferencia crítica entre los ejercicios reside en la lógica de control; mientras que el sensor de luminosidad requiere una inicialización simple, el sensor de presión introduce una lógica de recuperación de errores que obliga al código a tomar decisiones basadas en retroalimentación negativa, demostrando que en MMIO la CPU no solo recibe datos, sino que debe gestionar activamente la resiliencia del sistema frente a fallos de hardware. Desde una perspectiva técnica, el ejercicio del monitor de tensión arterial destaca la capacidad del MMIO para gestionar múltiples entradas de datos, tras un único evento de control. Al mapear TensionSistol y TensionDiastol, el programador puede recuperar un perfil fisiológico completo utilizando desplazamientos de memoria constantes tras veri-

ficar que el bit de TensionEstado ha cambiado a 1. Este diseño subraya por qué MIPS es eficiente en sistemas embebidos: permite que procedimientos como LeerLuminosidad o LeerPresion sean modulares y reutilizables, encapsulando la complejidad de las direcciones físicas, tras etiquetas simbólicas que facilitan la lectura del código y reducen la probabilidad de errores de direccionamiento en el bus.